

Documentation de réponse au cahier des charge

RABAH Soumaya & HOARAU Nolan
Projet SAÉ R3.02 - Onion Routing "Axolotl"
BUT 2 Réseaux et Télécommunications

DOCUMENTATION DE REPONSE AU CAHIER DES CHARGE.....	1
1. REPONSE AU CAHIER DES CHARGES.....	3
1.1 ÉLEMENTS IMPLEMENTES	3
<i>Gestion du réseau de routeurs :</i>	3
<i>Routage multi-sauts en oignon :</i>	3
<i>Chiffrement asymétrique RSA :</i>	3
<i>Interfaces graphiques PyQt6 :</i>	3
<i>Gestion multi-threads :</i>	3
<i>Déploiement distribué :</i>	3
<i>Double interface :</i>	3
<i>Fonctionnalités secondaires :</i>	4
1.2 ÉLEMENTS NON IMPLEMENTES	4
<i>Visualisation graphique de la topologie :</i>	4
<i>Statistiques avancées :</i>	4
<i>Chiffrement de bout en bout :</i>	4
<i>Découverte automatique des routeurs :</i>	4
<i>Résilience et redondance :</i>	4
1.3 FONCTIONNALITES BONUS	4
2. ARCHITECTURE ET STRUCTURE DU CODE	5
2.1 ARCHITECTURE GENERALE	5
2.2 STRUCTURE DES FICHIERS.....	5
2.3 MODULES PRINCIPAUX	5
<i>Master Server :</i>	5
<i>Routeur :</i>	5
<i>Client :</i>	5
2.4 BASE DE DONNEES	6

3. PROTOCOLE ET API.....	6
3.1 PROTOCOLE DE COMMUNICATION	6
<i>Enregistrement d'un routeur :</i>	6
<i>Enregistrement d'un client :</i>	6
<i>Demande de chemin :</i>	6
<i>Liste des utilisateurs :</i>	6
<i>Keep-alive :</i>	6
<i>Message en oignon :</i>	6
3.2 GESTION DES ERREURS	6
4. ALGORITHME DE CHIFFREMENT.....	7
4.1 PRINCIPE : RSA SIMPLIFIE.....	7
<i>Génération des clés :</i>	7
<i>Chiffrement :</i>	7
<i>Déchiffrement :</i>	7
4.2 CONSTRUCTION DU CHIFFREMENT EN OIGNON.....	7
4.3 FORCES ET FAIBLESSES.....	7
<i>Forces :</i>	7
<i>Faiblesses critiques :</i>	8
<i>Améliorations possibles :</i>	8
5. RAPPORT DE GESTION DE PROJET.....	8
5.1 REPARTITION DU TRAVAIL	8
5.2 METHODOLOGIE	9
5.3 DIFFICULTES RENCONTREES	9
<i>Problèmes techniques :</i>	9
<i>Problèmes organisationnels :</i>	9
5.4 RESPECT DU PLANNING.....	9
5.5 METRIQUES.....	9
5.6 BILAN	10
<i>Points positifs :</i>	10
<i>Axes d'amélioration :</i>	10
<i>Compétences développées :</i>	10
6. CONCLUSION.....	10

1. Réponse au Cahier des Charges

1.1 Éléments Implémentés

Notre projet a implémenté avec succès l'ensemble des fonctionnalités essentielles demandées dans le cahier des charges. Le système de communication anonyme est pleinement fonctionnel avec un serveur Master centralisé, des routeurs intermédiaires et des clients communicants.

Gestion du réseau de routeurs : Nous avons développé un système complet d'enregistrement des routeurs auprès du serveur Master. Chaque routeur reçoit un identifiant unique et des clés RSA sont générées et distribuées automatiquement. Les informations sont stockées de manière persistante dans une base de données MariaDB, et un mécanisme de désenregistrement propre a été implémenté pour gérer l'arrêt des routeurs.

Routage multi-sauts en oignon : L'implémentation du routage en oignon est complète et fonctionnelle. Les messages sont chiffrés en plusieurs couches successives, chaque routeur ne déchiffrant qu'une seule couche et ne connaissant que son voisin immédiat. Le système supporte un nombre variable de routeurs (de 1 à N couches), configurable par l'utilisateur.

Chiffrement asymétrique RSA : Bien que simplifié pour des raisons pédagogiques, notre implémentation RSA génère automatiquement des paires de clés publique/privée pour chaque routeur et client. Le chiffrement s'effectue caractère par caractère avec la clé publique, et le déchiffrement utilise la clé privée correspondante. Le Master distribue les clés publiques de manière sécurisée.

Interfaces graphiques PyQt6 : Nous avons développé deux interfaces graphiques complètes et modernes. L'interface Master propose une visualisation en temps réel des routeurs et clients connectés avec un système d'onglets (Logs et Connexions), des tableaux dynamiques mis à jour automatiquement, et une configuration personnalisable de l'IP et du port. L'interface Client offre une fenêtre de connexion intuitive, une zone de chat avec messages horodatés sous forme de bulles colorées, la sélection du destinataire via menu déroulant, le choix du nombre de couches de routage, et le rafraîchissement de la liste des utilisateurs. Les deux interfaces utilisent un design moderne avec la palette de couleurs Catpuccin.

Gestion multi-threads : Le système gère efficacement les connexions simultanées grâce à des threads séparés pour chaque connexion, un thread d'écoute des messages entrants, et un thread keep-alive pour maintenir la connexion avec le Master. Les interfaces graphiques sont non-bloquantes, permettant une expérience utilisateur fluide.

Déploiement distribué : Notre système supporte le déploiement sur plusieurs machines virtuelles avec une configuration IP/port personnalisable pour chaque composant, une validation des adresses et une vérification de disponibilité des ports.

Double interface : Nous proposons à la fois une interface en ligne de commande (CLI) et une interface graphique (GUI), sélectionnables au démarrage, assurant une compatibilité avec différents environnements d'utilisation.

Fonctionnalités secondaires : Le système intègre également une journalisation complète avec des logs horodatés affichés en temps réel, une base de données persistante avec quatre tables relationnelles (routeurs, utilisateurs, messages, routes), un nettoyage automatique au démarrage, et la gestion des états en ligne/hors ligne des utilisateurs.

1.2 Éléments Non Implémentés

Certaines fonctionnalités n'ont pas été implémentées par choix de priorisation ou contraintes de temps.

Visualisation graphique de la topologie : Nous n'avons pas développé de représentation graphique du réseau avec nœuds et connexions animés. Cette fonctionnalité aurait nécessité des bibliothèques graphiques complexes et un temps de développement conséquent pour un résultat non critique au fonctionnement du système.

Statistiques avancées : Aucun tableau de bord avec métriques détaillées (latence, nombre de messages, graphiques d'utilisation) n'a été implémenté. Nous avons préféré nous concentrer sur les fonctionnalités essentielles du routage en oignon.

Chiffrement de bout en bout : Le contenu du message n'est pas chiffré spécifiquement pour le destinataire final. Seul le routage est anonymisé, ce qui correspond au scope de base du routage en oignon tel que demandé dans le cahier des charges.

Découverte automatique des routeurs : Il n'existe pas de protocole de découverte dynamique des routeurs. L'enregistrement est manuel, conformément à notre architecture centralisée volontaire basée sur le Master.

Résilience et redondance : Le système ne propose pas de chemins alternatifs en cas de panne d'un routeur ni de mécanisme de retry automatique. Cette simplification permet de maintenir un prototype fonctionnel et compréhensible.

1.3 Fonctionnalités Bonus

Nous avons ajouté plusieurs fonctionnalités supplémentaires améliorant l'expérience utilisateur. La configuration est flexible avec la possibilité de personnaliser l'IP et le port de chaque composant via des fenêtres de configuration dédiées. La gestion des déconnexions est propre avec un désenregistrement automatique des routeurs et un nettoyage des ressources. Toutes les entrées utilisateur sont validées (formats IP, disponibilité des ports) avec des messages d'erreur explicites. Le design est moderne et cohérent avec un thème sombre, des polices monospace pour les logs, et des retours visuels clairs.

2. Architecture et Structure du Code

2.1 Architecture Générale

Notre projet suit une architecture client-serveur distribuée à trois niveaux hiérarchiques. Au sommet se trouve le serveur Master qui gère la topologie du réseau, distribue les clés RSA, coordonne les routeurs et maintient la base de données MariaDB. Au niveau intermédiaire, les routeurs reçoivent les messages, déchiffrent une couche et les transmettent au saut suivant. À la base, les clients envoient des messages en construisant le chiffrement en oignon et reçoivent les messages qui leur sont destinés.

2.2 Structure des Fichiers

Le code source est organisé dans le dossier `source/` contenant trois fichiers principaux : `master.py` (environ 900 lignes) qui implémente le serveur Master avec ses classes `MasterServer`, `MasterSignals`, `MasterWindow` et `MasterLoginWindow` ; `router.py` (environ 380 lignes) qui contient les fonctions d'enregistrement, déchiffrement RSA, gestion des connexions et serveur d'écoute ; et `client.py` (environ 950 lignes) avec les classes `ChatClient`, `LoginWindow`, `ChatWindow`, `MessageSignals` et la fonction `run_cli()` pour le mode terminal. Le fichier `requirements.txt` liste les dépendances Python nécessaires.

Le dossier `database/` contient le schéma SQL (`schema.sql`), la configuration de connexion (`config.py`) et un script de test (`test_db.py`).

2.3 Modules Principaux

Master Server : La classe `MasterServer` initialise le serveur avec les paramètres IP, port et mode GUI. Elle maintient en mémoire la liste des routeurs connectés, le dictionnaire des clients et leurs statuts en ligne. Les méthodes principales incluent `start()` pour démarrer le serveur et initialiser la base de données, `handle_router()` pour l'enregistrement des routeurs avec génération de clés RSA, `handle_client()` pour gérer les connexions clients et traiter leurs commandes (LIST, GET, PATH, PING, QUIT), et `request_path()` pour créer des chemins de routage en sélectionnant aléatoirement N routeurs disponibles.

Routeur : Le module routeur est composé de fonctions essentielles. La fonction `register()` établit la connexion avec le Master et reçoit l'ID et les clés RSA. La fonction `decrypt()` effectue le déchiffrement RSA caractère par caractère. La fonction `handle_connection()` traite les messages entrants en les convertissant en liste d'entiers, les déchiffre, extrait le prochain saut et transmet le payload. La fonction `start_server()` lance le serveur d'écoute sur le port configuré, et `unregister()` assure un désenregistrement propre lors de l'arrêt.

Client : La classe `ChatClient` gère toute la logique client. Les méthodes réseau comprennent `register()` pour la connexion au Master, `get_online_users()` et `get_user_info()` pour récupérer les informations des utilisateurs disponibles, et `request_path()` pour demander un chemin de routage. Les méthodes cryptographiques incluent `encrypt_message()` pour le chiffrement RSA et `build_onion()` pour construire les couches successives. Les méthodes de communication sont `send_message()` qui orchestre l'envoi complet, `listen_for_messages()` qui écoute en continu dans un thread séparé, et `keep_alive()` qui maintient la connexion active avec des PING réguliers.

2.4 Base de Données

Le schéma relationnel comprend quatre tables principales. La table `routers` stocke l'ID, l'IP, le port, les composantes des clés RSA (e , n , d) et les timestamps d'enregistrement. La table `users` contient le username, l'IP, le port, les clés publiques (e , n), un booléen `is_online` pour le statut et les timestamps. La table `messages` permet la journalisation optionnelle avec expéditeur, destinataire, message chiffré/déchiffré et chemin. La table `routes` conserve l'historique des routes créées.

Les opérations implémentées incluent la création automatique des tables, l'insertion des routeurs et utilisateurs, la mise à jour des statuts et timestamps, la suppression lors des désenregistrements, les requêtes `SELECT` pour rechercher les entités disponibles, et le nettoyage complet au démarrage du Master.

3. Protocole et API

3.1 Protocole de Communication

Notre système utilise TCP/IP avec des messages textuels encodés en UTF-8. Chaque type d'opération suit un format spécifique.

Enregistrement d'un routeur : Le routeur envoie d'abord "ROUTER", attend 500ms, puis envoie "IP;PORT". Le Master répond avec "ID;d;n" contenant l'identifiant et les clés privées.

Enregistrement d'un client : Le client envoie "CLIENT", attend 100ms, puis "username::ip::port". Le Master répond "OK:e:n" avec la clé publique.

Demande de chemin : Le client envoie "PATH:sender:nb_layers:target". Le Master répond avec la liste des routeurs séparés par "|" suivie de "||" et les informations du destinataire : "ip1;port1;e1;n1|ip2;port2;e2;n2||target_ip;target_port".

Liste des utilisateurs : Le client envoie "LIST" et reçoit "ONLINE:user1,user2,user3".

Keep-alive : Échange de "PING" et "PONG" toutes les 30 secondes pour maintenir la connexion active.

Message en oignon : Le message chiffré est transmis sous forme de liste d'entiers séparés par des virgules. À chaque routeur, une couche est déchiffrée et le reste est transmis au suivant jusqu'au destinataire final qui reçoit "sender:message" en clair.

3.2 Gestion des Erreurs

Le système implémente plusieurs codes d'erreur explicites : "ERROR:DB_CONNECTION" pour les problèmes de base de données, "ERROR:INVALID_FORMAT" et "ERROR:INVALID_DATA" pour les formats incorrects, "ERROR:TARGET_NOT_FOUND" quand le destinataire n'existe pas, "ERROR:NO_ROUTERS_AVAILABLE" s'il n'y a pas assez de routeurs, et "ERROR:UNKNOWN_COMMAND" pour les commandes non reconnues.

Les timeouts sont configurés à 10 secondes pour les connexions au Master, 5 secondes pour le forwarding entre routeurs, et 30 secondes pour le keep-alive. Les déconnexions sont gérées avec des try/except, un nettoyage systématique des ressources, et la mise à jour du statut en base de données.

4. Algorithme de Chiffrement

4.1 Principe : RSA Simplifié

Nous avons implémenté une version éducative du chiffrement RSA avec des nombres premiers réduits (entre 11 et 50). Cette simplification rend le système compréhensible et rapide à exécuter, mais totalement inadapté à un usage en production.

Génération des clés : L'algorithme choisit aléatoirement deux nombres premiers distincts p et q , calcule leur produit $n = p \times q$, puis l'indicatrice d'Euler $\phi(n) = (p-1) \times (q-1)$. L'exposant public e est choisi premier avec $\phi(n)$ (généralement $e = 3$ ou supérieur), et l'exposant privé d est calculé comme l'inverse modulaire de e modulo $\phi(n)$. Par exemple, avec $p = 17$ et $q = 23$, on obtient $n = 391$, $\phi(n) = 352$, $e = 3$ et $d = 235$.

Chiffrement : Chaque caractère du message est converti en code ASCII puis chiffré selon la formule $c = m^e \bmod n$. Le résultat est une liste d'entiers chiffrés.

Déchiffrement : Chaque entier chiffré est déchiffré selon $m = c^d \bmod n$, puis reconverti en caractère. Une gestion spéciale traite les valeurs hors de la plage Unicode valide.

4.2 Construction du Chiffrement en Oignon

Le message traverse N routeurs. Pour chaque routeur, une couche contient un header (adresse IP et port du prochain saut) suivi d'un séparateur "|" et du payload (qui peut être une couche chiffrée supplémentaire).

L'algorithme de construction procède de l'intérieur vers l'extérieur. Il démarre avec le message clair "username:message", puis pour chaque routeur en partant du dernier, il construit une couche avec le format "next_hop|current_payload", la chiffre avec la clé publique du routeur, et convertit le résultat en chaîne d'entiers séparés par des virgules. Le message final est l'oignon complet prêt à être envoyé au premier routeur.

4.3 Forces et Faiblesses

Forces : Notre implémentation garantit un bon anonymat de l'expéditeur car chaque routeur ne connaît que son voisin immédiat et le destinataire ne peut pas remonter jusqu'à l'expéditeur. La simplicité du RSA éducatif facilite la compréhension sans dépendances cryptographiques externes. L'architecture est modulaire avec un nombre de couches configurable et des routeurs ajoutables dynamiquement. Le code est pédagogique, clair et bien commenté.

Faiblesses critiques : La sécurité cryptographique est volontairement faible. Les nombres premiers trop petits (11-50) produisent des modules $n \leq 2500$, cassables par force brute en quelques secondes. Le chiffrement caractère par caractère est vulnérable à l'analyse de fréquence. L'absence de padding rend le système déterministe : le même message produit toujours le même chiffré, permettant des attaques par dictionnaire. **Ce système ne doit JAMAIS être utilisé en production.**

Les performances sont limitées par l'exponentiation modulaire répétée pour chaque caractère, entraînant une lenteur notable et une augmentation exponentielle de la taille des messages (multiplication par 3-4 à chaque couche). Le système ne propose pas de chiffrement de bout en bout : le dernier routeur voit le message en clair. Il manque également des mécanismes avancés comme l'authentification de l'expéditeur, les signatures numériques, la vérification d'intégrité et le Perfect Forward Secrecy.

Améliorations possibles : Pour un système de production, il faudrait utiliser des bibliothèques cryptographiques standards (comme `cryptography` en Python), augmenter la taille des clés à minimum 2048 bits, ajouter du padding OAEP, implémenter un chiffrement hybride (RSA pour l'échange de clés, AES-256-GCM pour le contenu), ajouter des signatures numériques et implémenter Perfect Forward Secrecy avec des échanges Diffie-Hellman éphémères.

5. Rapport de Gestion de Projet

5.1 Répartition du Travail

Soumaya RABAH (moi) s'est principalement occupée de l'infrastructure du projet en configurant le dépôt Git avec son système de branches, en structurant les dossiers, et en installant MariaDB avec les scripts de création de base de données. Sa contribution majeure concerne le développement complet des interfaces graphiques PyQt6 : l'interface Master avec ses fenêtres de configuration et de monitoring (onglets, tableaux, logs), et l'interface Client avec la fenêtre de connexion et l'interface de chat moderne. Elle a appliqué un design cohérent avec le thème Catppuccin et assuré l'intégration avec la logique backend via les signaux Qt. Elle a également rédigé l'essentiel de la documentation : planning préparatoire, guides utilisateur et d'installation, et sections du rapport final.

Nolan HOARAU a développé tout le cœur technique du système. Il a implémenté le chiffrement RSA complet avec la génération de clés, les fonctions de chiffrement/déchiffrement et la construction du système en oignon. Il a créé l'architecture réseau complète avec les sockets TCP, la gestion multi-threads, le protocole de communication personnalisé, et la gestion des timeouts et erreurs. Il a codé la logique métier du Master (enregistrement, distribution des clés, création de chemins), du Routeur (réception, déchiffrement, forwarding) et du Client (construction des messages, envoi, réception). Il a également développé le module d'accès à la base de données avec les requêtes CRUD et la gestion des transactions.

Tâches collaboratives : Nous avons travaillé ensemble sur l'architecture globale lors de sessions de brainstorming sur Discord, défini conjointement le protocole de communication et les interfaces entre modules. L'intégration entre les interfaces graphiques et la logique réseau a nécessité une collaboration étroite pour implémenter les signaux Qt, les callbacks et la gestion des threads non-bloquants. Les phases de tests (unitaires, intégration, déploiement multi-VM) et de débogage ont été réalisées en pair programming. Enfin, la documentation finale et la vidéo de démonstration sont le fruit d'un travail commun.

5.2 Méthodologie

Nous avons adopté une méthode agile adaptée avec des sprints courts de 5 à 7 jours : préparation (13-20/11), modules de base (21-30/11), Master (01-08/12), Routeur (09-13/12), Client (14-17/12), Tests (18-19/12), Documentation (20-28/12) et Finalisation (29-30/12). Des stand-ups quotidiens sur Discord (10-15 minutes) au format "Hier / Aujourd'hui / Blocages" nous permettaient de synchroniser nos avancements et résoudre rapidement les problèmes.

Nous utilisons Git/GitHub avec une stratégie de branches claire (main stable, dev pour l'intégration, feature/* pour les développements), des commits atomiques avec messages conventionnels (feat:, fix:, docs:, refactor:), et des pull requests avec review mutuelle. Discord servait de canal de communication principal avec des channels dédiés et des sessions de partage d'écran. Nos IDE (VS Code/PyCharm) étaient configurés avec des extensions pertinentes (Python, GitLens, MariaDB) et des linters pour maintenir la qualité du code.

5.3 Difficultés Rencontrées

Problèmes techniques : Plusieurs obstacles ont été surmontés. Les ports occupés lors des tests ont été résolus avec `SO_REUSEADDR`, la vérification de disponibilité avant bind, et un nettoyage systématique avec `atexit`. Les threads bloquants qui gelaient l'interface Qt ont nécessité l'implémentation de threads daemon, de signaux Qt pour la communication inter-threads, et de callbacks asynchrones. Les messages tronqués sur le réseau ont été corrigés en augmentant la taille du buffer de réception à 1MB. Les caractères non-ASCII mal déchiffrés ont été gérés avec une conversion modulo 256 pour les codes hors plage Unicode. La base de données avec des données fantômes a été nettoyée par un TRUNCATE systématique au démarrage. Les déconnexions non détectées ont été résolues par un thread keep-alive avec PING/PONG toutes les 30 secondes.

Problèmes organisationnels : Nos horaires parfois décalés ont été compensés par un planning partagé, des plages horaires communes définies, des commits fréquents et une documentation inline abondante. L'intégration de modules développés séparément a nécessité une définition claire des interfaces au départ et des tests d'intégration systématiques. L'apprentissage de PyQt6, nouveau pour nous deux, a demandé du temps mais la progression incrémentale et l'entraide en pair programming ont été efficaces.

5.4 Respect du Planning

Le planning a été globalement respecté avec une livraison dans les temps le 30 décembre. Un léger décalage progressif (+1 jour par phase) s'est accumulé jusqu'à la phase de tests (+2 jours), mais nous avons rattrapé ce retard en parallélisant la documentation avec le développement final. Sur 48 jours prévus, nous avons utilisé 48 jours, validant l'efficacité de notre organisation.

Les facteurs de succès incluent le découpage modulaire permettant le travail en parallèle, la communication efficace via les stand-ups quotidiens, la priorisation stricte des fonctionnalités essentielles, les tests continus dès le début, et la documentation maintenue au fil du développement.

5.5 Métriques

Le projet représente environ 2230 lignes de code Python (900 pour master.py, 950 pour client.py, 380 pour router.py), 80 lignes SQL et environ 1800 lignes de documentation. Nous avons réalisé 85

commits répartis équitablement (42 pour Soumaya, 43 pour Nolan), majoritairement des fonctionnalités (53%) et corrections de bugs (26%). Le temps de développement total est estimé à 150 heures (75h chacun).

5.6 Bilan

Points positifs : Notre collaboration a été efficace grâce à la complémentarité de nos compétences. L'architecture solide et modulaire facilite la maintenance et les évolutions futures. Nous avons livré toutes les fonctionnalités essentielles avec des interfaces soignées et une documentation complète. Les compétences acquises (PyQt6, routage en oignon, gestion de projet agile, Git collaboratif) sont substantielles.

Axes d'amélioration : Nous aurions pu écrire davantage de tests automatisés et mettre en place une intégration continue. La gestion des erreurs pourrait couvrir plus de cas limites. Les performances du chiffrement RSA sont limitées et pourraient bénéficier d'un système hybride. L'architecture centralisée présente un point de défaillance unique qui pourrait être résolu par une redondance du Master.

Compétences développées : Soumaya a acquis une expertise en design d'interfaces modernes, maîtrise de PyQt6, modélisation de bases de données, rédaction technique et coordination de projet. Nolan a développé ses compétences en cryptographie appliquée, programmation réseau avec sockets, architecture distribuée, intégration de bases de données et debugging bas niveau. Nous avons tous deux progressé en travail d'équipe, gestion des priorités, auto-formation et déploiement d'applications distribuées.

6. Conclusion

Le projet Axolotl - Onion Routing a été mené à bien avec succès. Nous avons développé un système de communication anonyme complet comprenant un serveur Master centralisé avec interface graphique, des routeurs capables de déchiffrer et transmettre, des clients avec interface moderne, un protocole de communication personnalisé, un chiffrement en oignon fonctionnel, une base de données persistante et un déploiement multi-VM validé.

L'objectif principal d'anonymat des communications est atteint : l'identité de l'expéditeur est masquée au destinataire et aucun routeur ne connaît le chemin complet. Notre système démontre concrètement les principes du routage en oignon similaires à Tor. Les objectifs secondaires (interfaces intuitives, architecture modulaire, documentation complète, respect des délais) sont également remplis.

Les limites actuelles incluent une cryptographie volontairement faible (usage éducatif uniquement), l'absence de chiffrement de bout en bout, une architecture centralisée et des performances limitées. Les évolutions futures possibles comprennent une cryptographie robuste (RSA-2048, AES-256), un chiffrement hybride, l'authentification et les signatures numériques, un Master distribué avec consensus, une interface web et le support de fichiers.

Ce projet nous a permis de concrétiser des concepts théoriques en une application fonctionnelle, de relever des défis techniques et organisationnels, et d'acquérir des compétences substantielles. Le résultat est un système éducatif démontrant efficacement les principes du routage en oignon, accompagné d'une documentation complète. Nous sommes fiers du travail accompli durant ces 48 jours de développement intensif.